Aide-mémoire : CREATE TABLE et contraintes dans PostgreSQL (v1)

Notations : mots-clés en MAJUSCULES, paramètres en minuscules, `[ ]` = facultatif, `|` = un choix parmi plusieurs, `...` = répétable.

1. Créer une table

```
CREATE TABLE [IF NOT EXISTS] nom_table (
    colonne type [contrainte_de_colonne ...],
    ...
    [CONSTRAINT nom] contrainte_de_table,
    ...
);
```

2. Contraintes de colonne

Contrainte	Effet
`NOT NULL`	Valeur obligatoire
`NULL`	Valeur facultative (comportement par défaut)
`DEFAULT valeur`	Valeur utilisée si l'`INSERT` n'en fournit pas
`UNIQUE`	Deux lignes ne peuvent pas avoir la même valeur
`PRIMARY KEY`	Clé primaire (implique `NOT NULL` et `UNIQUE`)
`CHECK (expression)`	La valeur doit vérifier la condition
`REFERENCES t (col)`	Clé étrangère sur une seule colonne
`GENERATED BY DEFAULT AS IDENTITY`	Numérotation automatique (1, 2, 3…), valeur explicite permise
`GENERATED ALWAYS AS IDENTITY`	Numérotation automatique, valeur explicite interdite

Une contrainte de colonne peut être nommée : `colonne type CONSTRAINT nom CHECK (...)`.

3. Contraintes de table

Écrites après les colonnes ; elles peuvent porter sur plusieurs colonnes.

```
[CONSTRAINT nom] PRIMARY KEY (col1 [, col2 ...])
[CONSTRAINT nom] UNIQUE (col1 [, col2 ...])
[CONSTRAINT nom] CHECK (expression)
[CONSTRAINT nom] FOREIGN KEY (col [, ...])
    REFERENCES autre_table (col [, ...])
    [ON DELETE action] [ON UPDATE action]
```

4. Actions des clés étrangères

Ce qui se passe quand la ligne référencée est supprimée (`ON DELETE`) ou sa clé modifiée (`ON UPDATE`) :

Action	Effet
`NO ACTION`	Par défaut : refus si des lignes y font référence (vérifié en fin d'instruction)
`RESTRICT`	Refus immédiat si des lignes y font référence
`CASCADE`	Les lignes qui y font référence sont supprimées (ou mises à jour) aussi
`SET NULL`	La clé étrangère des lignes concernées passe à `NULL`
`SET DEFAULT`	La clé étrangère reprend sa valeur par défaut

5. Expressions utiles dans un CHECK

Expression	Exemple
Comparaison	`prix > 0`, `date_fin >= date_debut`
Intervalle	`note BETWEEN 0 AND 20`
Liste de valeurs	`statut IN ('actif', 'inactif')`
Motif simple	`code LIKE 'FR%'` (`%` = n'importe quelle suite, `_` = un caractère)
Expression régulière	`cp ~ '^[0-9]{5}$'` (5 chiffres exactement)
Longueur	`char_length(nom) >= 2`
Combinaison	`AND`, `OR`, `NOT`, `IS NULL`, `IS NOT NULL`

Un `CHECK` ne porte que sur la ligne en cours : il ne peut pas consulter une autre table.

6. Types PostgreSQL courants

Type	Contenu
`smallint`	Entier de −32 768 à 32 767 (2 octets)
`integer`	Entier de −2 147 483 648 à 2 147 483 647 (4 octets)
`bigint`	Entier sur 8 octets
`numeric(p, s)`	Décimal exact : p chiffres dont s après la virgule (montants)
`varchar(n)`	Chaîne de longueur variable, au plus n caractères
`char(n)`	Chaîne de longueur fixe n, complétée par des espaces
`text`	Chaîne sans limite de longueur
`boolean`	`true` / `false`
`date`	Date sans heure, saisie `'AAAA-MM-JJ'`
`timestamp`	Date et heure, saisie `'AAAA-MM-JJ HH:MM:SS'`

Correspondance avec un dictionnaire de données

Dictionnaire	PostgreSQL
entier	`integer`, ou `smallint` si les valeurs restent petites
entier servant d'identifiant	`integer GENERATED BY DEFAULT AS IDENTITY` (ancienne écriture : `SERIAL`)
car(n)	`varchar(n)`
car(n) de longueur toujours identique	`char(n)`
date	`date`

Les types `UNSIGNED INT`, `UNSIGNED SMALLINT`, `TINYINT` viennent de MySQL : ils n'existent pas dans PostgreSQL. Pour interdire les négatifs : `CHECK (x >= 0)`.

7. Exemple complet

```
CREATE TABLE departement (
  id  integer GENERATED BY DEFAULT AS IDENTITY,
  nom varchar(30) NOT NULL,
  CONSTRAINT pk_departement PRIMARY KEY (id),
  CONSTRAINT uq_departement_nom UNIQUE (nom)
);

CREATE TABLE employe (
  id             integer
                 GENERATED BY DEFAULT AS IDENTITY,
  nom            varchar(30) NOT NULL,
  salaire        numeric(8, 2),
  date_embauche  date NOT NULL DEFAULT CURRENT_DATE,
  departement_id integer NOT NULL,
  CONSTRAINT pk_employe PRIMARY KEY (id),
  CONSTRAINT ck_employe_salaire CHECK (salaire > 0),
  CONSTRAINT fk_employe_departement
    FOREIGN KEY (departement_id)
    REFERENCES departement (id) ON DELETE RESTRICT
);
```

Une table référencée doit exister avant la table qui la référence : `departement` est créée en premier.

8. Modifier une table

```
ALTER TABLE t ADD COLUMN col type [contrainte ...];
ALTER TABLE t DROP COLUMN col;
ALTER TABLE t RENAME COLUMN ancien TO nouveau;
ALTER TABLE t RENAME TO nouveau_nom;
ALTER TABLE t ADD CONSTRAINT nom CHECK (expr);
ALTER TABLE t ADD CONSTRAINT nom
  FOREIGN KEY (col) REFERENCES t2 (col)
  ON DELETE CASCADE;
ALTER TABLE t DROP CONSTRAINT nom;
ALTER TABLE t ALTER COLUMN col SET DEFAULT valeur;
ALTER TABLE t ALTER COLUMN col DROP DEFAULT;
ALTER TABLE t ALTER COLUMN col SET NOT NULL;
ALTER TABLE t ALTER COLUMN col DROP NOT NULL;
ALTER TABLE t ALTER COLUMN col TYPE nouveau_type;
ALTER TABLE t ALTER COLUMN col
    ADD GENERATED BY DEFAULT AS IDENTITY;
```

Pour changer l'action `ON DELETE` d'une clé étrangère existante : `DROP CONSTRAINT`, puis `ADD CONSTRAINT` avec la nouvelle action.

9. Supprimer une table

```
DROP TABLE [IF EXISTS] t [, t2 ...] [CASCADE];
```

`CASCADE` supprime aussi les clés étrangères qui pointent vers la table. Sans lui, supprimez les tables dans l'ordre inverse de leur création.

10. Index

```
CREATE [UNIQUE] INDEX nom_index
  ON t (col [, ...]);
-- index partiel : seulement les lignes
-- qui vérifient la condition
CREATE [UNIQUE] INDEX nom_index ON t (col)
  WHERE condition;
DROP INDEX nom_index;
```

Un index `UNIQUE` partiel impose l'unicité seulement parmi les lignes qui vérifient la condition.

11. Commentaires

```
COMMENT ON TABLE t IS 'texte';
COMMENT ON COLUMN t.col IS 'texte';
```

Les commentaires sont stockés dans la base et visibles dans pgAdmin (propriétés de l'objet).

12. Préfixes de noms de contraintes

Préfixe	Usage	Exemple
`pk_`	Clé primaire	`pk_employe`
`fk_`	Clé étrangère	`fk_employe_departement`
`uq_`	Unicité	`uq_departement_nom`
`ck_`	Vérification	`ck_employe_salaire`
`idx_`	Index	`idx_employe_nom`